

Student Success Early-Signal & Academic Planning System

Final Project Technical Report

Rohit Cheruku

MSCS 2201 Artificial Intelligence

Professor: Dr. Fadi Muheidat

September 2026

Central question: *Can historical academic and engagement patterns provide useful early signals of student difficulty, and can AI generate a feasible academic plan that helps an advisor respond?*

Executive Summary

This project builds a tool to support academic advisors by combining two AI methods. The first part uses supervised machine learning to identify students who may need extra attention. The second part creates a two-term academic plan that follows course and degree requirements. The dataset is fully synthetic and includes about 2,200 students across four programs. For model development, I used 5,148 labeled training records from 1,716 students. The final test set contains 1,452 records from 484 different students, with no student overlap between the two sets. I first created a simple rule-based baseline, then compared Logistic Regression, Random Forest, and Gradient Boosting. Logistic Regression performed best overall on F1 and recall during cross-validation, so I selected it for the final evaluation.

On the final test set, Logistic Regression improved recall from 55.52% to 62.63% and F1 from 36.58% to 39.60% compared with the rule baseline. It found 20 more true intervention cases, which reduced false negatives from 125 to 105. The trade-off was 16 additional false positives. The fairness review showed similar recall across the three synthetic audit groups, although Audit_C had the highest false-positive rate at 43.52%. The academic planner also produced a valid, non-empty plan for all 12 required advisor cases, and every plan passed a separate validation check. Overall, the system is useful as a support tool, but the results are not strong enough to replace advisor judgment.

1. Project Goal and Data

I treated the project as two connected problems. First, the early-signal model estimates whether a student should be brought to an advisor's attention because the synthetic data indicates that intervention may be needed within the next eight weeks. Second, the planner recommends courses for the next two terms while following prerequisites, course availability, credit limits, and degree requirements. I kept these two parts separate because a risk prediction should not automatically lead to an academic decision.

The dataset is fully synthetic, so it does not contain real students, schools, courses, or academic records. The training set has 5,148 observations, the final test set has 1,452 observations, and the current-student file has 2,200 unlabeled students. About 20.26% of the training examples are positive cases, which means accuracy by itself would not be a very useful measure due to the imbalance. The training and test sets also contain different students, so the same student does not appear in both model development and final evaluation.

The model uses information such as prior academic performance, recent pass and failure history, current course load, assignment submission behavior, and recent assignment scores. Program and entry type are included as categorical features. Student_ID is left out because it is only an identifier. I also left out Observation_Term as a modeling choice because the current prediction term comes after the development terms, and I did not want the model to rely on term labels that may not generalize well to future data. Audit_Group is kept in a separate file and is never used to make predictions. It is only joined back later to compare model errors across the synthetic audit groups, as required by the project.

Patterns in the training data

A few patterns stood out in the training data. Assignment submission rate, GPA change, late assignments, recent course pass rate, LMS activity, days since the last LMS activity, and recent failed courses all showed some relationship with the target. I treated these as useful signals, not as proof that any one factor causes a student to need intervention. The later failure analysis is important because it shows that students can still need help even when the available indicators look positive, and the opposite can also happen.

2. Early-Signal Modeling

Rule-based baseline

I started with a simple rule-based baseline so there would be an easy-to-understand comparison for the machine-learning model. The baseline checks five warning signs and flags a student when at least two are present. The thresholds were chosen using the training data and were kept fixed before the final test evaluation.

Indicator	Rule
GPA decline	GPA change ≤ -0.20
Low recent pass rate	Pass rate < 0.80
Low assignment submission	Submission rate < 0.80
High late-assignment rate	Late rate > 0.25
LMS inactivity	Days since last activity ≥ 4

Machine-learning model comparison

For the machine-learning step, I used the same preprocessing setup for all three models. Numeric values were standardized, while program and entry type were converted into categorical model features. I compared Logistic Regression, Random Forest, and Gradient Boosting using five-fold cross-validation grouped by Student_ID. Grouping by student helped make sure records from the same student did not appear in both the training and validation portion of the same fold. The decision threshold stayed at 0.50, and the final test set was not used to choose the model or adjust the threshold. This follows the course idea of learning from labeled examples while keeping test data separate for a fair final evaluation (Muheidat, 2025; Russell & Norvig, 2021).

Model	Precision	Recall	F1	ROC-AUC
Logistic Regression	30.98%	62.61%	41.45%	67.50%
Random Forest	34.08%	38.06%	35.96%	65.81%
Gradient Boosting	31.67%	38.35%	34.69%	63.06%

Logistic Regression was selected because it had the best cross-validated F1 score and the highest recall of the three models. Random Forest had better precision and accuracy, but it missed many more positive cases. For this project, recall matters because a false negative means a student who may need support is not flagged. For that reason, I focused more on recall and F1 than on accuracy alone.

3. Final Evaluation and Fairness Review

After the model choice was finalized, I evaluated the rule baseline and Logistic Regression on modeling_test.csv. The machine-learning model improved recall by 7.1% and F1 by 3.0%, while accuracy changed very little. The confusion-matrix results make the difference easier to see. Logistic Regression found 176 true positives and missed 105

positive cases. The rule baseline found 156 true positives and missed 125. That means the model reduced missed intervention cases by 20, but it also increased false positives from 416 to 432.

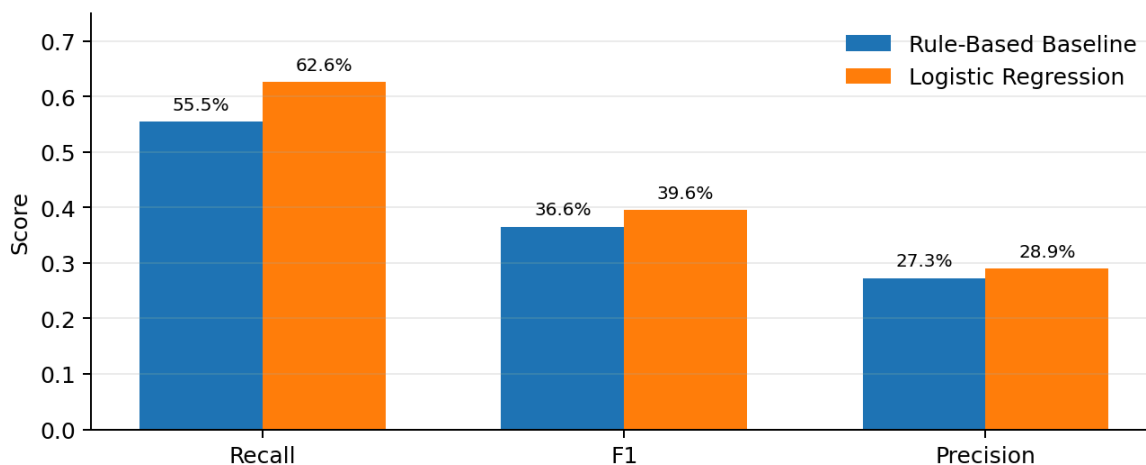


Figure 1. Baseline versus Logistic Regression on the final test set. The largest improvement is in recall.

Approach	Precision	Recall	F1	Accuracy	FN	FP
Rule baseline	27.27%	55.52%	36.58%	62.74%	125	416
Logistic Regression	28.95%	62.63%	39.60%	63.02%	105	432

Fairness and subgroup results

After the predictions were created, I joined them with the separate audit_groups.csv file to compare errors across Audit_A, Audit_B, and Audit_C. Recall was very similar across the groups: 62.60% for Audit_A, 63.27% for Audit_B, and 61.67% for Audit_C. The false-negative rates were also close. The biggest difference was the false-positive rate. Audit_C reached 43.52%, compared with 36.53% for Audit_A and 34.14% for Audit_B. So, even though recall looked similar overall, one group received more unnecessary flags.



Figure 2. Error rates across the synthetic audit groups. Audit_C has the highest false-positive rate.

These results should not be treated as proof of real-world demographic fairness. The audit groups are synthetic, and they were never used as model inputs. The purpose of this analysis is simply to check whether the model makes

different kinds of mistakes across groups. It shows why subgroup results are worth reviewing even when the overall metrics look reasonable.

4. Academic Planning with Constraints

The academic planner searches for a valid set of courses for T6_Spring and T7_Fall. I designed it around the same basic CSP ideas covered in class: choices, possible options, and rules that every valid solution must follow (Muheidat, 2026; Poole & Mackworth, 2023). Instead of using a separate optimization package, the program checks possible course combinations and keeps the plan that makes the most useful degree progress. Core requirements are prioritized first, followed by needed electives and then the number of degree-progress credits.

The planner can be described with the following CSP-style pieces:

- Variables / choices: which courses are selected for Spring and Fall
- Domains: the courses that are still needed and are available in the correct term.
- Prerequisites: required prerequisite courses must be completed before the dependent course. Completing both in the same term does not count.
- Completed courses: a course the student has already passed cannot be recommended again.
- Availability and credit limits: courses must be offered in that term, and the total credits must stay within the student's limit.
- Degree progress: each recommended course must count toward a remaining core or elective requirement.

A separate validator checks each plan after the planner creates it. This gives the project an extra check instead of assuming the generated plan is correct. All 12 required advisor cases passed validation, none of the plans were empty, and none broke the provided planning rules. This shows that the planner handled the supplied cases successfully, although it does not guarantee that every possible student record would have a valid two-term plan.

Case_02 prerequisite example: Program_A_C02 is recommended for Spring. If the student passes it, the course can satisfy the prerequisite for Program_A_C04 and Program_A_C05 in Fall. If the Spring course is not passed, the Fall plan needs to be reviewed again.

5. End-to-End Case Studies and Failure Analysis

Advisor case example: Case_06

Case_06 (Student_1248, Program_B) is a good example of the full workflow. Logistic Regression gave the student a 77.6% risk score, which is above the fixed 50% threshold, so the case was flagged for advisor review. The rule baseline also showed all five warning indicators. The main evidence shown by the model included seven days since the last LMS activity, a 68.0% assignment submission rate over four weeks, and a GPA change of -0.40. These are signals that influenced the model score. They are not explanations of why the student may need help.

For the same student, the planner recommended Program_B_C02 in Spring for 3 credits, followed by Program_B_C03 and Program_B_C05 in Fall for 6 credits. Program_B_C02 can satisfy the prerequisite for Program_B_C05 if the Spring course is passed successfully. The final plan passed the independent validation check, but it still requires an advisor to review it before any real academic decision is made.

Examples where the model was wrong

I also looked at cases where the model was clearly wrong. For the false-negative example, Student_1383 received only an 18.37% risk score even though the final outcome showed that intervention was needed. The available indicators actually looked fairly positive. The student had a GPA change of +0.74, an 86.7% recent pass rate, no failed courses or withdrawals, a 100% assignment submission rate, and zero days since the last LMS activity. This example shows that the dataset does not contain every factor that may matter.

The false-positive example shows the opposite problem. Student_0880 received a 90.29% risk score, but the final outcome showed that intervention was not needed. This student had several concerning signals, including a GPA change of -1.34, one failed course, one withdrawal, a 56.4% assignment submission rate, a 35.5% late-assignment rate, and an average assignment score of 58.5. Even with those warning signs, the outcome was negative. This is another reminder that the model's inputs are indicators, not causes or guarantees.

Why the errors matter: A false negative can delay support for a student who needs it. A false positive can use advisor time unnecessarily or create concern. For that reason, the model is used to support review rather than trigger automatic action.

6. Responsible AI, Limitations, and Research Questions

The system is intentionally designed to support an advisor, not replace one. Audit_Group and Student_ID are not used to make predictions. Risk scores are presented as early signals rather than diagnoses or explanations of a student's situation. Every generated academic plan requires advisor review. The system is also not allowed to contact students or register courses on its own. This human-review approach fits the course discussion of responsible AI and the importance of considering how AI systems affect people (Poole & Mackworth, 2023).

There are several important limitations. First, the entire dataset is synthetic, so the model's performance may not match what would happen at a real university. The positive class is also much smaller than the negative class, and the available academic and engagement data only gives a partial view of a student's situation. The project does not show that any feature causes a student to need intervention. The risk scores also should not be read as perfectly calibrated probabilities. In a real setting, an advisor would have other information that is not included here, such as instructor feedback, student goals, accommodations, scheduling issues, and direct conversations with the student.

Research question findings

RQ	Finding
RQ1	Logistic Regression improved recall from 55.52% to 62.63% and F1 from 36.58% to 39.60%. It produced 20 fewer false negatives, with 16 more false positives.
RQ2	Useful signals included assignment submission, GPA change, lateness, recent pass/failure history, and LMS activity. These are associations, not causes.
RQ3	Recall was similar across all three audit groups, but Audit_C had the highest false-positive rate at 43.52%.
RQ4	All 12 advisor cases received non-empty plans that passed validation and followed the required planning rules.
RQ5	Confident false-negative and false-positive examples show that model results still need human review.

Reproducibility

The repository includes commands to reproduce the main project steps. ``python main.py --run-required-pipeline`` runs the required backend in order, and ``python main.py --export-results`` creates the files used for the report and presentation. The professor-provided CSV files remain outside Git, while derived results and summaries are tracked. Safeguard tests also check restricted model features and key planning rules.

Conclusion

The final prototype gives a qualified yes to the project's main question. The academic and engagement data provides a useful early signal because Logistic Regression reduced the number of missed intervention cases compared with the rule baseline. The planner also generated valid two-term plans for all 12 supplied advisor cases. At the same time, the model still makes a substantial number of false-positive and false-negative errors, and the subgroup results are not identical. The course plans also depend on assumptions such as successfully completing Spring prerequisites. Because of these limitations, the best use of the system is as a transparent decision-support tool. It can help an advisor decide what to review and provide a feasible starting plan, while the advisor keeps full control over the final academic decision.

References

Muheidat, F. (2025). Module 8: Supervised Learning & Machine Learning [Course slides]. MSCS 2201 Artificial Intelligence.

Muheidat, F. (2026). Module 3: Constraint Satisfaction Problems (CSP) [Course slides]. MSCS 2201 Artificial Intelligence.

Poole, D. L., & Mackworth, A. K. (2023). Artificial intelligence: Foundations of computational agents (3rd ed.). Cambridge University Press.

Russell, S. J., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.

Final Project Option 6. (2026). Student Success Early-Signal & Academic Planning System — Machine Learning + Constraint Satisfaction | Optional Agentic AI [Course project instructions].